La librairie Seaborn

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Fondamentaux de Seaborn **3**

A. Qu'est-ce que Seaborn ? 3

B. Installation et configuration 4

C. Concepts de base 5

D. Premiers graphiques 5

E. Exercice : Quiz 6

III. Visualisations statistiques avancées **7**

A. Diagrammes de distribution 7

B. Relations entre les variables 8

C. Graphiques catégoriels 9

D. Graphiques multi-panneaux et multi-plots 10

E. Personnalisation des graphiques 11

F. Exercice : Quiz 13

IV. Essentiel **14**

V. Auto-évaluation **15**

A. Exercice 15

Solutions des exercices **15**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 60 minutes

Environnement de travail : PC ou Mac avec 8 Go de ram minimum

Pré requis : Bases en Python

Contexte

Dans le contexte de l'analyse de données, la visualisation est un outil essentiel pour comprendre, explorer et communiquer les informations contenues dans des ensembles de données. La bibliothèque Seaborn, développée pour Python, offre une interface simple et efficace pour créer des graphiques esthétiques et informatifs, particulièrement adaptés aux analyses statistiques. Seaborn s'intègre parfaitement avec les DataFrames de Pandas, ce qui en fait un choix privilégié pour les scientifiques de données, les analystes et les développeurs.

Ce cours est conçu pour fournir une compréhension pratique et approfondie de Seaborn, en commençant par son installation et ses fonctionnalités de base, puis en explorant des visualisations statistiques plus complexes. Les étudiants apprendront à exploiter les caractéristiques de Seaborn pour enrichir leurs analyses de données et structurer efficacement leurs projets de visualisation.

Objectifs pédagogiques :

- Apprendre à utiliser Seaborn pour créer des visualisations de données statistiques.
- Découvrir les fonctionnalités avancées de Seaborn pour l'analyse visuelle de données.

II. Fondamentaux de Seaborn

A. Qu'est-ce que Seaborn ?

Présentation de Seaborn

Seaborn est une bibliothèque de visualisation de données pour Python, conçue pour simplifier et améliorer les graphiques créés avec Matplotlib. Seaborn permet de générer des graphiques esthétiques et informatifs avec peu de code, en se spécialisant dans les visualisations statistiques. Elle est particulièrement utile pour explorer les corrélations entre variables, la distribution des données, et la création de graphiques complexes comme les matrices de corrélation ou les graphiques à facettes.

Rappel de Pandas

Pandas est une bibliothèque centrale pour la manipulation et l'analyse de données en Python, optimisée pour gérer des ensembles de données structurés sous forme de tables, appelées DataFrames. Un DataFrame est une structure de données bidimensionnelle, similaire à une feuille de calcul ou une table de base de données, avec des étiquettes pour les lignes et colonnes, permettant de stocker et traiter facilement des ensembles de données.

Pandas permet d'importer, de nettoyer, de manipuler et d'analyser des données provenant de diverses sources, notamment des fichiers CSV (Comma-Separated Values). Lorsqu'un fichier CSV est chargé dans Pandas, il est transformé en un DataFrame, ce qui simplifie le travail avec les données. Les colonnes d'un DataFrame représentent les différentes variables d'un ensemble de données, tandis que chaque ligne représente une observation individuelle.

Utiliser une feuille CSV avec Pandas et Seaborn

Le format CSV est l'un des moyens les plus communs pour stocker des données tabulaires et est facilement manipulable avec Pandas. Voici les étapes pour lire et utiliser un fichier CSV avec Pandas et Seaborn :

Méthode **Lecture du fichier CSV**

Avec Pandas, lire un fichier CSV est simple et rapide. La fonction `pd.read_csv()` permet de charger le fichier et de le convertir en DataFrame.

```
1 import pandas as pd
2 data = pd.read_csv('nom_du_fichier.csv')
```

Méthode **Exploration et manipulation du Dataframe**

Seaborn est conçu pour fonctionner avec les DataFrames de Pandas, ce qui permet d'accéder facilement aux différentes variables du fichier CSV pour créer des graphiques. Par exemple, pour visualiser la distribution d'une variable appelée age, il suffit de faire :

```
1 import seaborn as sns import matplotlib.pyplot as plt
2 sns.histplot(data['age'], kde=True)
3 plt.title('Distribution des âges')
4  plt.show()
```

Avantages de Seaborn par rapport à d'autres bibliothèques

Seaborn se distingue par sa **facilité d'utilisation** qui permet de créer rapidement des visualisations sophistiquées. En une seule ligne de code, comme `sns.histplot(data['variable'], kde=True)`, on peut générer des graphiques avec des configurations automatiques, économisant du temps tout en assurant clarté et précision grâce à des paramètres par défaut judicieusement choisis pour l'analyse statistique.

La bibliothèque est aussi appréciée pour son esthétique ; **elle produit par défaut des visualisations élégantes** avec des palettes de couleurs qui améliorent la lisibilité, idéales pour les rapports et les présentations. Les détails tels que les lignes de grille discrètes et les styles de texte bien équilibrés contribuent à la clarté visuelle.

Seaborn fonctionne en harmonie avec les DataFrames de Pandas, facilitant l'accès et la manipulation des données par leur nom de colonne, ce qui simplifie la transition de l'analyse brute à la visualisation.

B. Installation et configuration

Méthode **Comment installer Seaborn ?**

Seaborn peut être installé en utilisant pip, le gestionnaire de paquets Python. La commande à exécuter dans le terminal ou la console de commande est la suivante :

```
1 $ pip install seaborn
```

Il est également possible de l'installer via **conda**, un autre gestionnaire de paquets populaire, surtout pour les utilisateurs qui utilisent Anaconda pour gérer leurs environnements Python. Remplacer pip par panda dans la commande

Méthode **Configuration de l'environnement de travail**

Pour utiliser Seaborn, il est important de configurer correctement votre environnement de développement.

Dans votre script ou notebook, importez Seaborn ainsi que les autres bibliothèques nécessaires, typiquement Pandas pour la manipulation des données et Matplotlib pour des ajustements supplémentaires sur les graphiques.

Tous droits réservés à STUDI - Reproduction interdite

```python
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

Exemple **Tester l'installation**

Pour vérifier que Seaborn est correctement installé et fonctionnel, vous pouvez essayer de créer un graphique simple, comme un diagramme de dispersion :

```python
sns.scatterplot(x=[1, 2, 3], y=[3, 2, 1])
plt.show()
```

Cela vous permettra de démarrer avec Seaborn et de préparer le terrain pour des explorations de données visuelles plus avancées.

C. Concepts de base

Présentation des types de données pouvant être utilisées avec Seaborn

Seaborn est optimisé pour fonctionner avec des structures de données tabulaires, et il est particulièrement efficace lorsqu'il est utilisé avec des DataFrames de Pandas. Voici les principaux types de données que vous pouvez utiliser avec Seaborn :

- **DataFrames de Pandas :** c'est le type de données le plus communément utilisé avec Seaborn pour la visualisation, car il permet de stocker des données sous forme de tableaux avec des étiquettes pour les lignes et les colonnes. Seaborn interagit directement avec ces DataFrames, facilitant l'accès aux différentes variables pour la création de graphiques.
- **Arrays NumPy :** utiliser des arrays NumPy, en particulier pour des tâches où les données sont numériques et le label des axes moins important.
- **Listes Python :** pour des graphiques simples, des listes de valeurs Python peuvent également être utilisées, bien que pour des analyses plus complexes et structurées, l'utilisation de DataFrames soit préférable.

Seaborn est conçu pour interpréter intuitivement les structures de données complexes, rendant l'exploration préliminaire et l'analyse statistique des données directes et efficaces.

D. Premiers graphiques

Création de graphiques simples pour explorer les données

L'exploration visuelle des données est un premier pas essentiel dans l'analyse statistique. Seaborn facilite cette étape avec des fonctions qui simplifient la création de graphiques. Voici ci-dessous, deux types de graphiques que vous pouvez créer pour commencer, à savoir, l'histogramme et le diagramme de dispersion.

Exemple **Histogrammes**

Utilisez histplot pour visualiser la distribution d'une variable continue. L'histogramme est un outil de base pour comprendre la distribution centrale et la dispersion des données.

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Création d'un DataFrame de démonstration
data = pd.DataFrame({
   'age': [23, 45, 56, 78, 21, 34, 49, 50, 20, 31, 73, 24, 58, 46, 35, 62]
})

# Tracé de l'histogramme avec estimation de la densité
```

Tous droits réservés à STUDI - Reproduction interdite

```
sns.histplot(data['age'], kde=True)
plt.title('Distribution des âges')
plt.show()
```

Exemple **Diagrammes de dispersion**

Utilisez `scatterplot` pour examiner les relations entre deux variables continues. Ce type de graphique est essentiel pour détecter les corrélations, les tendances ou les anomalies.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = pd.DataFrame({
   'height': [170, 180, 160, 175, 165, 185, 155, 190, 175, 170],
   'weight': [65, 80, 55, 70, 60, 90, 50, 85, 75, 65]
})

# Création d'un scatter plot utilisant seaborn
sns.scatterplot(x='height', y='weight', data=data)
plt.title('Relation entre la taille et le poids')
plt.xlabel('Taille (cm)')
plt.ylabel('Poids (kg)')
plt.show()
```

Ces graphiques simples sont les pierres angulaires de l'analyse de données et fournissent une base solide pour des explorations plus détaillées et complexes à mesure que vous progressez avec Seaborn. Ils sont également un excellent moyen de communiquer rapidement et efficacement les propriétés statistiques des données analysées.

E. Exercice : Quiz

[solution n°1 p.17]

Question 1

Qu'est-ce que Seaborn ?

- Une bibliothèque Python pour la manipulation de données
- Une bibliothèque de visualisation de données pour Python
- Un module pour la création d'applications web

Question 2

À quoi sert Pandas dans l'utilisation de Seaborn ?

- À créer des graphiques plus esthétiques
- À manipuler et analyser des données tabulaires
- À installer Seaborn

Question 3

Quelle méthode permet de lire un fichier CSV avec Pandas ?

- pd.read_csv()
- pd.import_csv()
- sns.read_csv()

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quel graphique Seaborn est recommandé pour visualiser la distribution d'une variable continue ?

- sns.scatterplot()
- sns.histplot()
- sns.lineplot()

Question 5

Quel est l'avantage d'utiliser Seaborn par rapport à d'autres bibliothèques de visualisation ?

- Il est plus rapide que d'autres bibliothèques
- Il est compatible uniquement avec les données au format CSV
- Il simplifie la création de graphiques statistiques et offre des visuels esthétiques par défaut

III. Visualisations statistiques avancées

A. Diagrammes de distribution

Les diagrammes de distribution permettent d'analyser et de visualiser la distribution des données numériques. Voici quelques fonctions de Seaborn spécifiques pour cela :

`kdeplot` : trace la courbe d'estimation de densité de kernel, qui est utile pour visualiser la distribution continue d'une variable unidimensionnelle sans les contraintes des bacs utilisés dans les histogrammes.

Exemple **kdeplot**

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt


data = pd.DataFrame({
   'income': np.random.normal(50000, 15000, 1000)  # Données fictives de revenu
})

# Affichage du graphique de densité
sns.kdeplot(data['income'], shade=True)
plt.title('Distribution du revenu')
plt.show()
```

`histplot` que nous avons vu dans la partie 1 sert à faire des diagrammes de distribution, notez que `histplot` dans les versions récentes de Seaborn(remplace `distplot`) : permet de combiner un histogramme avec une courbe de Densité kernel (KDE).

Tous droits réservés à STUDI - Reproduction interdite

B. Relations entre les variables

La compréhension des relations entre différentes variables dans un ensemble de données est cruciale pour l'analyse statistique et la prise de décision basée sur les données. Identifier comment les variables interagissent entre elles peut révéler des corrélations, des tendances, ou des modèles de comportement qui ne sont pas évidents lorsque les variables sont considérées isolément.

Seaborn, en tant que bibliothèque de visualisation de données pour Python, propose plusieurs outils graphiques qui aident à explorer et à visualiser ces relations. Ces graphiques peuvent illustrer non seulement la nature de la relation entre deux variables continues-par exemple, si elles varient ensemble de manière linéaire, exponentielle ou aléatoire-mais aussi comment cette relation est influencée par d'autres variables catégorielles.

Méthode **Scatterplot**

`Scatterplot` est Idéal pour visualiser la relation entre deux variables continues. Il montre comment les valeurs d'une variable varient avec une autre.

Exemple

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = pd.DataFrame({
   'age': [25, 30, 35, 40, 45, 50, 55, 60],
   'income': [50000, 60000, 70000, 80000, 90000, 100000, 110000, 120000]
})

# Utilisation de Seaborn pour créer un scatter plot
sns.scatterplot(x='age', y='income', data=data)
plt.title('Relation entre âge et revenu')

plt.show()
```

Méthode **Pairplot()**

Utile pour visualiser les relations entre toutes les combinaisons de plusieurs paires de variables dans un DataFrame, affichant des graphiques de dispersion pour les interactions et des histogrammes ou KDEs pour les distributions univariées.

Exemple

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Création d'un DataFrame avec des données de démonstration pour 'age', 'income' et 'height'
data = pd.DataFrame({
   'age': [25, 30, 35, 40, 45, 50, 55, 60, 65, 70],
   'income': [50000, 60000, 65000, 70000, 75000, 80000, 85000, 90000, 95000, 100000],
   'height': [165, 170, 175, 180, 185, 190, 195, 200, 205, 210]
})

# Utilisation de seaborn pour créer un pairplot qui montre les relations par paires entre
'age', 'income' et 'height'
sns.pairplot(data[['age', 'income', 'height']])
```


Tous droits réservés à STUDI - Reproduction interdite

```
plt.show()
```

Méthode **Jointplot**

`Joinplot` combine des diagrammes de dispersion ou des hexbins avec des histogrammes pour fournir des aperçus détaillés de la relation bivariée entre deux variables.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data_from_csv = pd.read_csv('data.csv')

sns.joinplot(data=data_from_csv, x='age', y='salaire', color='black')
plt.suptitle(t: "Relation entre l'âge et le salaire", y= 0.9)
```

C. Graphiques catégoriels

Pour comparer les distributions dans différentes catégories, Seaborn propose plusieurs graphiques spécialisés :

Méthode **Boxplot**

Montre la répartition d'une variable quantitative en catégories en utilisant les quartiles de la distribution et les valeurs extrêmes. Très utile pour détecter les valeurs aberrantes.

Exemple

```

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = pd.DataFrame({
   'gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
   'income': [70000, 80000, 60000, 85000, 90000, 75000, 65000, 95000, 55000, 90000]
})

# Utilisation de seaborn pour créer un boxplot qui montre la distribution du revenu par genre
sns.boxplot(x='gender', y='income', data=data)
plt.title('Distribution du revenu par genre')
plt.show()
```

Méthode **Violinplot**

`Violinplot` combine les aspects d'un boxplot et d'un KDE pour fournir des informations riches sur la distribution des données en catégories.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = pd.DataFrame({
    'gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
    'income': [70000, 80000, 60000, 85000, 90000, 75000, 65000, 95000, 55000, 90000]
})

# Utilisation de seaborn pour créer un violin plot qui montre la distribution du revenu par genre
sns.violinplot(x='gender', y='income', data=data)
plt.title('Violin plot du revenu par genre')
plt.show()
```

Méthode **Swarmplot**

Affiche tous les points de données sans chevauchement, ce qui peut donner une meilleure représentation de la distribution des valeurs, particulièrement en combinaison avec un `boxplot` ou `violinplot`.

Exemple

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = pd.DataFrame({
    'gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
    'income': [70000, 80000, 60000, 85000, 90000, 75000, 65000, 95000, 55000, 90000]
})

# Utilisation de seaborn pour créer un swarm plot qui montre la distribution du revenu par genre
# avec des points noirs semi-transparents
sns.swarmplot(x='gender', y='income', data=data, color='k', alpha=0.5)
plt.show()
```

Chacun de ces graphiques a ses propres forces dans le contexte d'une analyse de données, permettant aux analystes de choisir l'outil le plus approprié en fonction de leur objectif spécifique et de la nature de leurs données.

D. Graphiques multi-panneaux et multi-plots

Seaborn offre d'excellentes fonctionnalités pour créer des visualisations complexes en utilisant des grilles de graphiques, ce qui permet d'analyser des ensembles de données sous plusieurs perspectives à la fois. Voici deux fonctions importantes pour cela :

Méthode **FacetGrid**

Cet outil permet de créer des grilles qui facilitent la visualisation de distributions de variables et les relations entre plusieurs variables avec des conditions de sous-ensembles. Il est particulièrement utile pour explorer rapidement de grandes quantités de données et observer les effets de différentes variables catégorielles.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
g = sns.FacetGrid(tips, col="time", row="smoker")
g.map(sns.scatterplot, "total_bill", "tip")
plt.show()
```

Méthode **PairGrid**

Semblable à `pairplot`, mais avec plus de flexibilité, `PairGrid` permet de tracer des relations entre des paires de variables dans un ensemble de données. Contrairement à pairplot, il permet de définir des graphiques différents pour la diagonale, le triangle supérieur et le triangle inférieur de la grille.

Exemple

```
import seaborn as sns
import matplotlib.pyplot as plt

# Supposons que 'tips' soit un DataFrame qui vient de seaborn
tips = sns.load_dataset('tips')

# Création d'un PairGrid
g = sns.PairGrid(tips)
g.map_upper(sns.scatterplot)
g.map_lower(sns.kdeplot, color="C0")  # Utilise 'color' au lieu de 'colors'
g.map_diag(sns.histplot, kde=True, color='k')  # Active le KDE et définit la couleur

plt.show()
```

E. Personnalisation des graphiques

Seaborn offre plusieurs options pour personnaliser les graphiques, rendant vos analyses de données non seulement plus informatives mais aussi visuellement attrayantes.

Seaborn permet de modifier facilement les palettes de couleurs pour s'adapter à différentes présentations ou préférences visuelles. Utiliser `sns.set_palette()` pour définir une palette de couleurs globale pour tous les graphiques ou passer l'argument palette directement aux fonctions de tracé pour une personnalisation spécifique.

```
Ex: sns.set_palette("pastel") # Définit la palette "pastel" comme palette globale
```

Seaborn inclut plusieurs thèmes prédéfinis qui peuvent être utilisés pour styliser les graphiques. Le changement de thème peut être effectué avec la fonction `sns.set_theme()`, qui permet de contrôler des aspects tels que le style de fond, la palette de couleurs, et plus.

```
Ex : sns.set_theme(style="whitegrid") # Applique un fond à grille blanche
```

Pour les graphiques contenant de nombreuses variables ou des plages de données larges, il est crucial d'ajuster les échelles des axes et de modifier les légendes pour améliorer la clarté.

```
# Exemple de modification de l'échelle des axes et d'ajout de légendes
ax = sns.scatterplot(x="total_bill", y="tip", data=tips)
ax.set(xscale="log", xlim=(1, 100), ylim=(0, 10), xlabel='Total Bill (Log Scale)',
ylabel='Tip')
plt.legend(title='Tip Amount')  # Ajoute une légende avec un titre
plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

Exemple **Modification de l'échelle des axes et d'ajout de légendes**

```
import seaborn as sns
import matplotlib.pyplot as plt
# Chargement du jeu de données 'tips' de Seaborn
tips = sns.load_dataset('tips')

# Création d'un scatterplot avec catégorisation par 'day'
ax = sns.scatterplot(x="total_bill", y="tip", hue="day", data=tips)

# Configuration des échelles des axes, des limites et des étiquettes des axes
ax.set(xscale="log", xlim=(1, 100), ylim=(0, 10),
       xlabel='Total Bill (Log Scale)', ylabel='Tip')

# Ajout d'une légende avec un titre
plt.legend(title='Day of the Week')

# Affichage du graphique
plt.show()
```

Explications du code :

- **Importation des bibliothèques** : nous importons seaborn et matplotlib.pyplot, essentiels pour la création et la personnalisation des graphiques.
- **Chargement des données** : le DataFrame tips est chargé directement depuis la bibliothèque Seaborn, qui contient des données sur les pourboires laissés par les clients dans un restaurant, y compris le total de la facture (total_bill) et le jour de la semaine (day).
- **Création du scatterplot** : un scatterplot est créé avec une catégorisation par day en utilisant l'argument hue. Cela permet à chaque jour d'être représenté par une couleur différente sur le graphique.
- **Configuration des axes** : l'axe des x est mis à l'échelle logarithmique pour mieux visualiser les données étendues. Les limites des axes (xlim et ylim) et les étiquettes des axes (xlabel et ylabel) sont également définies.
- **Légende** : une légende est ajoutée avec le titre "Day of the Week" pour aider à identifier les couleurs associées à chaque jour.
- **Affichage** : plt.show() est utilisé pour afficher le graphique.

```
ax = sns.scatterplot(x="total_bill", y="tip", data=tips)
ax.set(xscale="log", xlim=(1, 100), ylim=(0, 10), xlabel='Total Bill (Log Scale)',
ylabel='Tip')
plt.legend(title='Tip Amount')
plt.show()
```

Ces techniques avancées permettent de raffiner les visualisations pour une communication efficace des analyses de données, adaptant les graphiques aux besoins spécifiques des audiences ou des objectifs de recherche.

Ajustement des axes et des légendes

L'adaptation des axes et des légendes est cruciale pour une communication claire des résultats analytiques via des visualisations graphiques. Choisir une échelle d'axe appropriée permet d'améliorer significativement la compréhension des variations et des tendances dans les données. Cette adaptation peut radicalement transformer la représentation des données, mettant ainsi en évidence les relations clés entre les variables.

Par ailleurs, ajuster les limites des axes est essentiel pour définir les plages de valeurs à afficher, ce qui focalise l'attention sur des zones particulières des données. Cette focalisation aide à mettre en lumière les aspects les plus pertinents de l'analyse.

Tous droits réservés à STUDI - Reproduction interdite

Les légendes, quant à elles, rendent les graphiques auto-explicatifs. Elles clarifient ce que représentent les différentes couleurs, formes, et autres éléments graphiques. En personnalisant soigneusement les légendes, on guide intuitivement le spectateur à travers les éléments visuels du graphique, facilitant ainsi une compréhension rapide et approfondie des informations présentées.

Méthode **Set_yscale et set_xscale**

Afin de changer les échelles on peut utiliser les méthodes : `set_yscale et set_xscale`. Pour Ajuster les limites d'axes les méthodes à utilisés sont `set_xlim` et `set_ylim` et pour personnaliser les légendes `plt.legend()` qui vous permet d'ajouter des légendes et de les personnaliser avec des titres, des positions, et des styles pour correspondre au contexte de votre analyse.

Exemple

```
import ssl

ssl._create_default_https_context = ssl._create_unverified_context

import seaborn as sns
import matplotlib.pyplot as plt
# Chargement des données
data = sns.load_dataset('diamonds')
# Création d'un scatter plot avec seaborn
plt.figure(figsize=(10, 6))
ax = sns.scatterplot(x='carat', y='price', hue='cut', data=data.sample(1000))
# Ajustement des échelles d'axes
ax.set(xscale="log", yscale="log")  # Mise à l'échelle logarithmique pour les axes x et y
# Ajustement des limites d'axes
ax.set_xlim(0.2, 3)  # Limite de carats
ax.set_ylim(100, 20000)  # Limite de prix
# Configuration de la légende
plt.legend(title='Quality of Cut', loc='upper left', fontsize='small', title_fontsize='13',
frameon=True, shadow=True)
# Affichage du graphique
plt.title('Relation between Carat and Price of Diamonds')
plt.xlabel('Carat (Log Scale)')
plt.ylabel('Price (Log Scale)')
plt.show()
```

Explications :

- Un scatter plot montre la relation entre le carat et le prix des diamants, avec des données catégorisées par la qualité de la taille ('cut').
- Les échelles logarithmiques sur les axes x et y facilitent la visualisation des données qui varient sur une large gamme de valeurs.
- Les limites sont ajustées pour se concentrer sur les diamants de taille courante et les prix correspondants.
- La légende est configurée avec un titre, positionnée pour éviter d'occulter les données, et stylisée pour être claire et informative.

F. Exercice : Quiz

[solution n°2 p.18]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Pourquoi est-il essentiel d'ajuster les axes et les légendes dans les visualisations de données

- Pour éviter que les graphiques ne soient tronqués
- Pour rendre les graphiques plus lisibles et mettre en évidence les informations essentielles
- Pour diminuer la taille du graphique

Question 2

Quel type de graphique Seaborn est utilisé pour visualiser une estimation de densité de distribution ?

- sns.kdeplot()
- sns.scatterplot()
- sns.boxplot()

Question 3

Quelle fonction Seaborn remplace distplot pour combiner un histogramme avec une courbe KDE ?

- sns.histplot()
- sns.pairplot()
- sns.violinplot()

Question 4

Quelle fonction Seaborn permet de visualiser toutes les relations par paires dans un DataFrame ?

- sns.scatterplot()
- sns.pairplot()
- sns.jointplot()

Question 5

Comment visualiser une relation bivariée entre deux variables avec Seaborn tout en affichant des histogrammes sur les axes ?

- sns.violinplot()
- sns.jointplot()
- sns.boxplot()

IV. Essentiel

Dans cette leçon, nous avons découvert Seaborn, une bibliothèque de visualisation de données pour Python, spécialement conçue pour simplifier et enrichir la création de graphiques statistiques. Seaborn s'intègre parfaitement avec Pandas, ce qui permet de manipuler des données sous forme de DataFrames avant de les visualiser.

Seaborn propose des outils puissants pour explorer les relations entre variables. Des fonctions avancées permettent de combiner des diagrammes de dispersion avec des histogrammes pour une exploration bivariée complète.Il est également possible de comparer des distributions catégorielles ou de créer des grilles multi-panneaux

Tous droits réservés à STUDI - Reproduction interdite

La personnalisation est également au cœur de Seaborn. Ajuster les couleurs et les styles pour améliorer l'esthétique des graphiques. Modifier les échelles d'axes et personnaliser les légendes pour renforcer la lisibilité et la pertinence des visualisations.

V. Auto-évaluation

A. Exercice

Création d'un histogramme avec courbe de densité kernel

Vous êtes analyste de données dans une entreprise qui souhaite mieux comprendre la distribution des âges de ses employés pour ajuster ses programmes de formation. L'entreprise vous a fourni un fichier contenant les âges de ses employés et vous demande de visualiser cette distribution pour identifier les tendances d'âge prédominantes.

Question 1

[solution n°3 p.19]

Votre mission est de créer un histogramme avec une courbe de Densité Kernel (KDE) pour visualiser la distribution des âges des employés de l'entreprise. (créer une data fictive qui se nomme donnees-ages..csv pour l'exercice)

Personnalisation d'un graphique de distribution

Après avoir créé un histogramme initial pour visualiser la distribution des âges des employés, vous êtes maintenant chargé d'améliorer ce graphique pour une présentation au conseil d'administration. L'objectif est de rendre le graphique plus lisible et visuellement attrayant en personnalisant différents éléments graphiques.

Question 2

[solution n°4 p.19]

Améliorez le graphique en appliquant les techniques de personnalisation apprises en cours.

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 6 Solution n°1

Question 1

Qu'est-ce que Seaborn ?

- ○ Une bibliothèque Python pour la manipulation de données
- ◉ Une bibliothèque de visualisation de données pour Python
- ○ Un module pour la création d'applications web

Seaborn est une bibliothèque pour Python, spécialisée dans les graphiques statistiques.

Question 2

À quoi sert Pandas dans l'utilisation de Seaborn ?

- ○ À créer des graphiques plus esthétiques
- ◉ À manipuler et analyser des données tabulaires
- ○ À installer Seaborn

Pandas est utilisé pour manipuler et analyser des données sous forme de DataFrames, ce qui simplifie l'importation de la préparation des données pour les graphiques Seaborn.

Question 3

Quelle méthode permet de lire un fichier CSV avec Pandas ?

- ◉ pd.read_csv()
- ○ pd.import_csv()
- ○ sns.read_csv()

La fonction pd.read_csv() permet de charger un fichier CSV dans un DataFrame.

Question 4

Quel graphique Seaborn est recommandé pour visualiser la distribution d'une variable continue ?

- ○ sns.scatterplot()
- ◉ sns.histplot()
- ○ sns.lineplot()

sns.histplot() sert à visualiser la distribution d'une variable continue sous forme d'histogramme, et peut inclure une courbe de densité pour enrichir l'analyse.

Question 5

Quel est l'avantage d'utiliser Seaborn par rapport à d'autres bibliothèques de visualisation ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Il est plus rapide que d'autres bibliothèques
- ○ Il est compatible uniquement avec les données au format CSV
- ◉ Il simplifie la création de graphiques statistiques et offre des visuels esthétiques par défaut

Exercice p. 13 **Solution n°2**

Question 1

Pourquoi est-il essentiel d'ajuster les axes et les légendes dans les visualisations de données

- ○ Pour éviter que les graphiques ne soient tronqués
- ◉ Pour rendre les graphiques plus lisibles et mettre en évidence les informations essentielles
- ○ Pour diminuer la taille du graphique

Q Ajuster les axes et les légendes améliore la lisibilité, permet de focaliser l'attention sur les zones pertinentes du graphique.

Question 2

Quel type de graphique Seaborn est utilisé pour visualiser une estimation de densité de distribution ?

- ◉ sns.kdeplot()
- ○ sns.scatterplot()
- ○ sns.boxplot()

Q sns.kdeplot() permet de tracer une estimation de densité de kernel.

Question 3

Quelle fonction Seaborn remplace distplot pour combiner un histogramme avec une courbe KDE ?

- ◉ sns.histplot()
- ○ sns.pairplot()
- ○ sns.violinplot()

Q sns.histplot() est la fonction utilisée dans les versions récentes de Seaborn pour créer des histogrammes avec une option pour superposer une estimation de densité KDE.

Question 4

Quelle fonction Seaborn permet de visualiser toutes les relations par paires dans un DataFrame ?

- ○ sns.scatterplot()
- ◉ sns.pairplot()
- ○ sns.jointplot()

Q sns.pairplot() crée une matrice de graphiques montrant les relations entre chaque paire de variables dans le DataFrame.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Comment visualiser une relation bivariée entre deux variables avec Seaborn tout en affichant des histogrammes sur les axes ?

- ○ sns.violinplot()
- ◉ sns.jointplot()
- ○ sns.boxplot()

sns.jointplot() combine des diagrammes de dispersion ou des hexbins avec des histogrammes, ce qui permet une exploration détaillée des relations bivariées entre deux variables.

p. 15 **Solution n°3**

```
1 import seaborn as sns
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 data = pd.read_csv('donnees_ages.csv')
6
7 sns.histplot(data['Age'], kde=True)
8
9 plt.show()
```

p. 15 **Solution n°4**

```
1 import seaborn as sns
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 data = pd.read_csv('donnees_ages.csv')
6
7 sns.set_theme(style="whitegrid")
8 sns.set_palette("pastel")
9
10 sns.histplot(data['Age'], kde=True)
11
12 plt.title("Distribution des Ages")
13 plt.xlabel('Generation')
14 plt.ylabel('Densité')
15
16 plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite